Log of the homework

* fully handmade document, no AI generation

Legend	
Prompts	**Bold**
My intakes on the outcome of the prompts	Normal

Baseline

I did not do any task like this since university, only worked on UI test automation projects and CI/CD pipeline development/integration, so this is what I chose as a base setup for the project:

- Claude with Pro subscription, VS Code and GitHub
    - Baseline: Sonnet 5 with High effort

Important prompts used during the homework

Day 1 – Setting up the baseline

As a test automation engineer, I have a task to develop a basic library web application with the testing and observability in focus to prove my skillset in my area of interest.

The base instructions for the project are the following:

"The goal of this coding challenge is to assess your ability to develop a RESTful web service, implement a simple database interaction, and write unit and integration tests, using Python, Golang, Java or NodeJS.

The application should be a Library Management System with basic functionality."

I'd like to use Java since I have the most experience with that, but I'm open for all the other possibilities.

First, based on the information bellow, I need the answer for this:

- **Which coding language and framework should I use to build this project, with keeping in mind the following:**
    - **Functional requirements:**
        - **Book Endpoints:**
            - **List Books: Retrieve a list of all available books**
            - **Add Book: Add a new book to the library**
            - **Borrow Book: Borrow a book by specifying the book ID and borrower ID**
        - **Borrower Endpoints:**
            - **Create Borrower: Register a new borrower**
            - **Get Borrower: Retrieve details of a specific borrower**

        - **Borrowed Books: Retrieve the list of books borrowed by a specific borrower**
    - **Non-Functional Requirements**
        - **Database:**
            - **Use any in-memory database (e.g., SQLite, H2) or external database (e.g., MySQL, PostgreSQL) of your choice**
            - **Ensure the application can run locally with the database configured appropriately**
        - **Observability**
            - **Instrument your application with at least one OpenTelemetry metric and one span**
    - **Testing Requirements**
        - **Unit Tests:**
            - **Write unit tests for the main components of your application**
            - **Use any testing framework of your choice**
    - **Deployment Instructions:**
        - **Local Deployment:**
            - **The application should be able to run locally**

Language/Framework

- Java 21 + Spring Boot 3.x
- spring-boot-starter-web (REST)
- spring-boot-starter-data-jpa (persistence)

Database

- H2, in-memory, via Spring Data JPA
- Entities: Book, Borrower, Loan (join entity for borrow records)

Observability

- OpenTelemetry Java SDK (or opentelemetry-spring-boot-starter)
- 1 custom span: around the borrow-book operation
- 1 custom metric via Micrometer: e.g. books.borrowed.count
- Export via OTLP to console/logging (no external collector needed)

Testing

- JUnit 5 + Mockito — unit tests (service layer, mocked repos)
- Spring Boot Test + MockMvc — integration tests (@SpringBootTest, @WebMvcTest, @DataJpaTest) against real H2 DB

Rationale

- Leverages existing Java expertise
- Spring Boot conventions (layered architecture, @Transactional, test slices) directly support demonstrating REST design, DB interaction, and test rigor

- Alternatives (Go, Node, Python) considered but weaker fit for showcasing typed domain modeling + test/observability depth under time constraints

Scaffold the project, and detail me each part of it here to better understand the structure and the way-of-working so I can demonstrate it easier in the future.

Here the project base structure and code was created, but my computer lacked JDK and Maven installs, so I had to do those manually and also resolve some issues with them (my PC was fully reinstalled a couple weeks ago (girlfriend's Planet Zoo 2 started to lag and I had to fix that), so now I have to set up everything from the bottom codingwise. This version was fully without UI, just the pure business logic and the API tests for that added, so I had to create something to show on the *http://localhost:8080*.

I want to have a functioning web application with a simple but user friendly UI where I can set up the user flows:

- **The website should function as an admin page, where I as the librarian can add, edit borrowers and books, pair borrowed books to borrowers and unpair them, also delete Books and Borrowers.**
- **Instrument a base UI for this, not yet wired together with the business logic and the database.**

At this point, the base webpage was created but not yet integrated with the local DB.

Rewire the solution to use the real database set up for this project, but in the meanwhile also make sure all these scenarios are covered:

- **Book Endpoints: Books have the following datas: Title(string), Author(string), Year of publication(integer), Edition(string), Book ID(string, auto generated, structure: lowercase, "_" connected ID of the book with the pattern like this: author_title_yearofpublication_edition)**
    1. **List Books: Retrieve a list of all available books, also be able to filter them by book id, title or author. (also show ID)**
        - **Add Book: Add a new book to the library (ask for all the details listed above except book id, generate that one from the provided information).**
        - **Borrow Book: Borrow a book by specifying the book ID and borrower ID.**
    2. **Borrower Endpoints: Borrowers have the following data: Name(string, can contain upper and lower characters), borrower ID(automatically generated string, structure: lowercase, "_" connected ID of the book with the pattern like this: name_dateofbirth), Date of birth(date), Address(String)**
        - **Create Borrower: Register a new borrower with all the information above.**
        - **Get Borrower: Retrieve details of a specific borrower (also show borrower ID).**
        - **Borrowed Books: Retrieve the list of books borrowed by a specific borrower.**

Here I decided to proceed with the little bit extended feature set, just to be more fitting for me.

After all the changes were done, I checked all the unit and integration tests ran and also the tested the span. I wanted to close up the 1st iteration here with a test database creation and a README update, so I ran these:

Create a base database with 10 mocked borrowers and 10 mocked books which will be pushed in the repository too.

- **Update the README file based on the current progress:**
  - **Technical stack and tools used**
  - **Detailed step by step instruction how to set up and run the project locally**
  - **How to check back the result of the unit and integration tests**
  - **How to check back the result of the span**

Notes to myself at the end of day1:

- UI is broken at some points, tables etc. **X**
- Redo the UI style to hungarian 2000s library page **X**
- remove my name from the folder structure **X**
- remove the **library**, **example** and the **com** from the folder structure **X**
- since a library can have multiple of the same books, and quantity to the book details **X**
- be able to loan out a book until the quantity reaches 0, same book can be added to different borrowers until the quantity does not run out **X**
- 1 borrower can only borrow the same book in 1 instance, the different edition does not matter **X**

Day 2 – Finalizing the project

The UI is very simple and very AI like. Please create a similar UI like this, which represent an older hungarian library's website: https://vfmk.hu/verseghy-konyvtar/v-szolgaltatasok/adatbazisok/ Do not consent the fields on the website, and don't change anything on the business logic now, just adapt the look of it.

After some finetunes, the original AI generated UI was changed to the Hungarian style.

Since a library can have multiple of the same books, add quantity to the book details which represents as an integer how many copies are available from the same book. Be able to loan out a book until the quantity of it reaches 0, but the same book can be added only to different borrowers until the quantity does run out. A borrower can only borrow the same book multiple times, but at the same time can have only 1 of it.

After testing the features, some fine tuning propmts

The "Borrower x already has an active loan for book y" error text's placing is bad, show it under the table at the center of the page not the top right corner.

The books table UI is broken. The button's padding in the last column is not right, all the other colums align well, but the button's column is shorter than the others, adjust it.

The width of the header and the tables are not the same. Make the tables on the pages as wide as the header parts (which have the add buttons inside).

Add this function:

- **If you select a borrower in the Borrowers table, open down a section under it where the loaned books of the borrower are listed. If the borrower has no borrowed books, show a placeholder which states the borrower has no loans currently.**

Remove the filter by ID function from the Books page. Also, move the other 2 Filter fields out of the table body of the listed book and place them above the table.

Notes to myself at the end of day2:

- Review the code in full manually **X**
- Review it to make it sure it aligns with the requirements doc. **X**
- Check the unit tests and the span if they are needed/add value or not. **X**
- add human readable test report generation **X**

Day 3 – Polishing, reviewing and testing

I have an official document which contains the requirements against this project as a job interview base for a technical round.

From the perspective of an employer who will make a judgement on the created project during a technical interview, check the requirements and the created project and tell me what is done and what's not done from the requirements. The one who interviews for the job is a senior/staff test automation engineer.

I looked on the outcomes and made some manual adjustments.

Can a tool be integrated into the project which generates an easily human readable version of the unit, integration tests runs and the outcome of the span?

After this I used Allure for the reporting, but skipped the span part cause it would have required a more bigger change.

Fully working **curl** for testing on Windows with Powershell:

```
curl.exe --% -X POST http://localhost:8080/api/books/robert_c_martin_clean_code_2008_1st/borrow -H "Content-Type: application/json" -d "{\"borrowerId\":\"jane_doe_19900512\"}"
```

Final notes

1. I only worked on the main branch. On a project like this which is not long living and has no MR structure I had no intention to do all the features/bit of changes on feature branches, adapt some features into integration branches and then merge them with quality gate checks and reviews into main. On a corporate environment of course I'd adapt for the development structure of my team and company and do all changes like I mentioned above.
2. I could have commited more in smaller batches, that was an error on my end.

3. I was never a frontend developer or a backend developer, and I either never worked with API tests and spans before just UI automation, so this was a learning opportunity for me too.